TACT - Testcase-driven Automatic Code Translation

Keagan Chern
UC Berkeley
kchern2@berkeley.edu

Sahil Bhatia
UC Berkeley
sahilbhatia@berkeley.edu

Alvin Cheung
UC Berkeley
akcheung@berkeley.edu

1 Introduction

Modern deep learning research frequently depends on custom kernels written in domain-specific languages (DSLs) to fully exploit accelerator performance. One notable example is Triton, a DSL engineered for high efficiency on NVIDIA GPUs. However, because Triton is tightly coupled with CUDA, it cannot run directly on alternative accelerators such as Intel's Gaudi or other non-CUDA platforms. Being able to run on alternative accelerators is important because it enables the possibility to leverage cost and power efficiency advantages, overcome hardware availability constraints, and avoid dependence on a single vendor's ecosystem. Crucially, different accelerators are optimized for different workload characteristics, so portability allows researchers to select the hardware that offers the best performance for their specific task. This leads to a significant gap: researchers and engineers proficient in one DSL may not want to deploy to new environments, since doing so may require learning an entirely different programming model.

Recent research has begun exploring the use of LLMs to automate kernel generation and optimization across frameworks and hardware. KernelBench [9] and TritonBench [7] both evaluate how effectively LLMs can produce performant and correct GPU kernels by translating from high level PyTorch code to low level CUDA or Triton implementations. These studies show that while LLMs can generate syntactically valid kernels, they often fall short of human-optimized performance due to limited hardware awareness and tuning. Building on these insights, our work takes the reverse direction, focusing instead on translating existing Triton kernels back into functionally equivalent PyTorch implementations. This approach emphasizes portability and correctness across heterogeneous hardware backends rather than low level synthesis for a single platform.

To address this challenge, we propose a testcase driven code translation approach called TACT that automatically translates Triton kernels into equivalent PyTorch implementations using Large Language Models(LLMs). Traditional compiler development requires substantial time and engineering effort, particularly when adapting to new hardware targets with distinct execution models. LLMs are particularly well suited for this task because of their ability to generalize across programming languages and frameworks.

Triton itself is designed to give developers fine-grained control over GPU execution. This enables them to write custom kernels that can outperform more generic frameworks implementations for certain workloads. Kernels in Triton are written using Python-like syntax but are compiled down to highly optimized GPU code that leverages CUDA primitives.

By contrast, PyTorch provides a higher level programming model in which developers write models in Python using tensor ops, and the framework automatically dispatches these operations to software backends optimized for different hardware. Backends such as TorchDynamo first capture user code and convert it into an intermediate graph representation, while TorchInductor lowers this graph into efficient kernels. With NVIDIA GPUs, these efficient kernels are generated in Triton, providing highly optimized CUDA execution. Through this design, PyTorch serves as a flexible entry point for researchers who want strong performance without being locked into a single low level DSL such as Triton.

Thus, PyTorch serves as a common denominator: when paired with torch.compile(), backends can lower PyTorch into optimized kernels (Triton on NVIDIA GPUs). By converting Triton kernels into PyTorch code, we extend their usability to hardware that does not support Triton, such as Intel's Gaudi, while preserving both functional correctness and equivalence of outputs.

2 TACT Architecture Overview

```python
#actual kernel for softmax
@triton.jit
def test_kernel(output_ptr, input_ptr, input_row_stride, output_row_stride, n_rows, n_cols, BLOCK_SIZE: tl.constexpr,
                num_stages: tl.constexpr):
    row_start = tl.program_id(0)
    row_step = tl.num_programs(0)
    for row_idx in tl.range(row_start, n_rows, row_step, num_stages=num_stages):
        row_start_ptr = input_ptr + row_idx * input_row_stride
        col_offsets = tl.arange(0, BLOCK_SIZE)
        input_ptrs = row_start_ptr + col_offsets
        mask = col_offsets < n_cols
        row = tl.load(input_ptrs, mask=mask, other=-float('inf'))
        row_minus_max = row - tl.max(row, axis=0)
        numerator = tl.exp(row_minus_max)
        denominator = tl.sum(numerator, axis=0)
        softmax_output = numerator / denominator
        output_row_start_ptr = output_ptr + row_idx * output_row_stride
        output_ptrs = output_row_start_ptr + col_offsets
        tl.store(output_ptrs, softmax_output, mask=mask)
```

(a) Softmax kernel written in Triton

```python
def test_pytorch(x: torch.Tensor) -> torch.Tensor:
    x_shifted = x - x.max(dim=1, keepdim=True).values
    numerator = torch.exp(x_shifted)
    return numerator / numerator.sum(dim=1, keepdim=True)
```

(b) Softmax written in PyTorch

```python
pytorch_output_small = test_pytorch(input_small)
assert torch.allclose(pytorch_output_small, output_small, rtol=1e-5, atol=1e-5), "Small test case failed"

pytorch_output_medium = test_pytorch(input_medium)
assert torch.allclose(pytorch_output_medium, output_medium, rtol=1e-5, atol=1e-5), "Medium test case failed"

pytorch_output_large = test_pytorch(input_large)
assert torch.allclose(pytorch_output_large, output_large, rtol=1e-5, atol=1e-5), "Large test case failed"
```

(c) generated Softmax test for PyTorch code

```python
properties = driver.active.utils.get_device_properties(
    torch.cuda.current_device()
)
NUM_SM = properties["multiprocessor_count"]
NUM_REGS = properties["max_num_regs"]
SIZE_SMEM = properties["max_shared_mem"]
WARP_SIZE = properties["warpSize"]
target = triton.runtime.driver.active.get_current_target()
kernels = {}
#driver for the triton kernel

def test(x):
    n_rows, n_cols = x.shape
    BLOCK_SIZE = triton.next_power_of_2(n_cols)
    num_warps = 8
    num_stages = 4 if SIZE_SMEM > 200000 else 2
    y = torch.empty_like(x)
    kernel = test_kernel.warmup(y, x, x.stride(0), y.stride(0),
    ↪  n_rows, n_cols, BLOCK_SIZE=BLOCK_SIZE,
                                num_stages=num_stages,
                                ↪  num_warps=num_warps,
                                ↪  grid=(1, ))
    kernel._init_handles()
    n_regs = kernel.n_regs
    size_smem = kernel.metadata.shared
    occupancy = NUM_REGS // (n_regs * WARP_SIZE * num_warps)
    occupancy = min(occupancy, SIZE_SMEM // size_smem)
    num_programs = NUM_SM * occupancy
    num_programs = min(num_programs, n_rows)
    kernel[(num_programs, 1, 1)](y, x, x.stride(0), y.stride(0),
    ↪  n_rows, n_cols, BLOCK_SIZE, num_stages)
    return y

```

(d) Softmax driver written in Triton

```python
def run_test_case(x):
    output = test(x)
    return x, output

test_cases = []

x_small = torch.randn(128, 64, device='cuda')
input_small, output_small = run_test_case(x_small)
test_cases.append(("small", input_small, output_small))

x_medium = torch.randn(512, 256, device='cuda')
input_medium, output_medium = run_test_case(x_medium)
test_cases.append(("medium", input_medium, output_medium))

x_large = torch.randn(2048, 1024, device='cuda')
input_large, output_large = run_test_case(x_large)
test_cases.append(("large", input_large, output_large))
```

(e) generated Softmax test for triton code

Figure 1: End-to-end example Tact framework on softmax

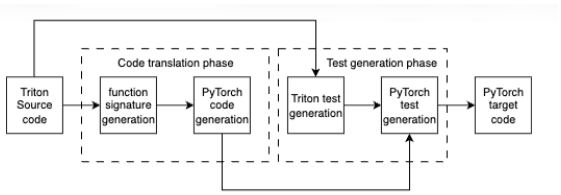

Figure 2: TACT architecture overview

The TACT framework employs a two-phase architecture for translating Triton kernels into functionally equivalent PyTorch implementations. As shown in Figure 2, the workflow consist of: (1) Code Translation Phase, where we translate the Triton kernel to PyTorch using LLMs, and (2) Test Validation Phase, where we generate and execute test cases to verify the correctness of the translation. The key innovation of this approach lies in its test-driven validation methodology: rather than assuming the LLM translation is correct straight from the model, TACT systematically generates test cases that verifies correctness between the source Triton kernel and the translated PyTorch implementation.

To illustrate TACT's operation, we use the softmax function as our end-to-end example. Softmax is a fundamental operation in deep learning that converts a vector of real numbers into a probability distribution. This is done by first subtracting the maximum value from all inputs to prevent numerical overflow, then exponentiating the shifted values. Each exponentiated value is divided by the sum of all exponentials, producing a normalized probability distribution that sums to one. The mathematical notation is shown here:

$$\text{softmax}(x_i) = \frac{e^{x_i - \max(x)}}{\sum_j e^{x_j - \max(x)}}$$

The code translation phase converts the Triton kernel into PyTorch code through a two-step process: The LLM first generates only the PyTorch function signature, translating Triton's pointer-based interface to PyTorch's tensor-based API. Triton kernels operate at the pointer level while PyTorch uses tensor-based inputs and outputs, generating a clear function signature ensures the model correctly interprets how data flows in and out. For the softmax example in Figure 1b, the LLM first generates the PyTorch function signature, `def test_pytorch(x: torch.Tensor) -> torch.Tensor`, which defines the functional interface and clarifies how data enters and exits the operation with an input tensor and an output tensor. This intermediate step also provides a convenient sanity check to verify that the input and output specifications align with implementation of softmax. Once this signature is generated, the model proceeds to the second step, generating the complete PyTorch implementation that reproduces the logic of the original Triton kernel(Figure 1c).

After the translation phase, TACT proceeds to the test case phase, which verifies the correctness of the generated PyTorch implementation. In this phase, TACT first produces a Triton-based test harness using the original Triton kernel and its corresponding driver code (Figure 1d), as shown in Figure 1e. The generated code defines a set of input tensors and computes the corresponding outputs by invoking the softmax function. To avoid potential bias during LLM translation, the original function name is replaced with test, though the underlying logic remains identical to softmax. Once the Triton test cases are generated (Figure 1e), TACT automatically constructs an equivalent PyTorch test harness (Figure 1d) that executes the translated function on the same inputs and compares the outputs within a predefined numerical tolerance. This automated validation ensures that the translated PyTorch code is functionally equivalent to the original Triton implementation. If the PyTorch program fails validation by not passing the generated

test cases, TACT automatically restarts both the code generation and test generation phases. This iterative sampling process works by generating many independent samples and uses our verification phase to select a correct one.[2] As demonstrated later in the evaluation, this limit was never reached in practice.

3 Code Translation

The objective of this phase is the translation of the source Triton to target PyTorch. Our code translation phases consist of two phases: 1) Function signature generation, and 2) Pytorch code generation. The LLM first generates only the PyTorch function signature before producing the full computation body. Triton kernels typically expose a low-level, pointer interface that mirrors GPU memory access patterns. Each input argument often represents a raw pointer to device memory, accompanied by explicit stride and dimension parameters. For example, in the softmax kernel shown in Figure 1a, the function `test_kernel` receives pointers such as `input_ptr` and `output_ptr`, along with their corresponding row strides and dimension variables (`n_rows, n_cols, BLOCK_SIZE`). This design allows developers to exercise fine-grained control over how data is loaded, stored, and iterated across GPU threads, enabling optimizations tailored to specific hardware configurations. However, it also couples the kernel's logic tightly to hardware specific addressing, and makes such code difficult to reason about semantically with higher-level frameworks like PyTorch that operate on structured tensor abstractions rather than raw memory pointers. As shown in Figure 3, the function signature generation prompt explicitly instructs the model to remove Triton-specific constructs and to produce a clean PyTorch-style function that returns values directly.

Once the function signature is generated, the second stage generates the full PyTorch implementation by translating the core computation logic from the source Triton kernel. In this step, the LLM is prompted with both the previously generated function signature and the original Triton source code, as shown in Figure 4. The prompt instructs the model to rewrite only the computational portion of the Triton program, exclude any low-level operations such as memory loading, storing, and blocking instructions, and to express it as a functionally equivalent PyTorch function. By constraining the LLM to reuse the given function signature without introducing additional parameters, the resulting translation preserves the functional interface while converting the kernel's pointer-based operations into tensor-level computations that can be executed directly within PyTorch.

4 Test-case Generation

The test case phase consists of two steps aimed at generating validation tests for both the Triton and PyTorch implementations to ensure output correctness. The first step focuses on constructing tests for the Triton kernel, which is particularly important when the test cases is not provided as part of the original source code. To accomplish this, TACT provides a specialized prompt to the LLM, instructing it to generate randomized input tensors for the given function. These automatically generated inputs are then used to execute the Triton kernel and record its outputs, forming the reference data against which the translated PyTorch implementation is later verified. Note that the inputs for this step consist solely

```
  """
Our task is to rewrite the given `triton` program to `pytorch`. First step in the translation is to come up with a
function signature for the `pytorch program`.
    #Instructions:
    1. Change the output pointers in the triton program to return values in the pytorch program.
    2. Do not include triton specific parameters for blocking and parallelization.
    3. The `pytorch` program should have a return type. Return only the function signature.
    ```
    #triton program
    @triton.jit
    {source_code}
    ```
"""
```

Figure 3: PyTorch function signature generation prompt

```
"""
Your task is to rewrite the given `triton` program in `PyTorch`. When rewriting to `Pytorch` ignore all the `loading`,
`storing` of data to the memory and `blocking` instructions. Rewrite only the computation part of the program as
a `PyTorch` kernel. Do not use any for loops for rewriting. The rewritten program should be functionally
equivalent to the `triton` program. The rewritten program should use the provided function signature.
```
    #triton program
    {source_code}
    ```
    ```
    Use the following template to rewrite the program in PyTorch. Do not add any additional parameters to the function signature or to the function body. The function
    signature should be exactly the same as the one provided below.
    ```
    #function signature
    {signature}
"""
```

Figure 4: PyTorch function signature generation prompt

of the Triton source code and its corresponding driver function, allowing it to run in parallel with the code generation phase since all required inputs are already available.

Figure 5 presents the prompt used for test case generation, which contains four key instructions identified as essential for reliable validation. The first instruction requires that the generated code include three distinct test cases. In principle, no finite set of test inputs can formally prove that two programs are semantically equivalent. However, random testing with concrete inputs is a practical and widely used approach for validating numerical kernels, where correctness primarily depends on numerical accuracy. We chose three input tensors of different sizes to provide simple yet categorical coverage of input variability, but this number of random inputs can increase by editing the prompt. Across more than 100 benchmarked TorchInductor kernels evaluated later in Section 5, 33 required more than one TACT iteration to pass validation; in all of these cases, none of the three test cases succeeded which indicates that no cases of partial correctness were observed. Among these, 5

```
"""
Your task is to take the following triton program and create test cases for it. We want to verify that it has an
output equivalent to a translated pytorch program.

    # **Instructions:**
    1. Generate random input tensors of small, medium, and large shapes for testing. A maximum of 3 cases.
    2. The test cases should call the specific driver function to invoke the Triton kernel. The given Test function
    format correctly runs the kernel and should not be changed
    3. Store and return the input tensors for future use in the pytorch program.
    4. Execute the test cases


    import triton
    # Triton implementation

    @triton.jit
    {triton_source_code}

    ```
    # Test Function format

    {driver_function_format}
"""
```

Figure 5: PyTorch function signature generation prompt

```
"""
You are an expert in deep learning programming frameworks

    Your task is to generate test cases to capture the output of the `pytorch` translation of the given `triton` program for comparison.


    # **Instructions:**
    1. You must run the same tensor inputs as that of the given triton testing code, and do not generate new inputs. You have access to all the triton test variables and naming. Imagine that the Triton test is ran before the pytorch test code that is generated and still in context
    2. execute the test cases and make sure the output of pytorch and triton is equivalent
    3. Do not have any other text besides the pytorch tests


    ```
    import torch

    # Pytorch implementation

    {pytorch_translation}

    # triton test function

    {triton_test}

"""
```

Figure 6: PyTorch function signature generation prompt

exhibited mismatched outputs, 21 encountered runtime errors, and 7 failed during compilation.

The second instruction ensures that the generated test code calls the existing driver function if one is provided in the source Triton program. This was necessary because the LLM often produced a new driver function instead of using the existing one without this instruction.

The third instruction specifies that both the input and output tensors must be stored after test execution. Without this step, the generated test code would execute but fail to preserve the data needed for subsequent PyTorch validation.

Finally, the fourth instruction enforces that the generated test cases are executed immediately rather than merely defined. This guarantees that the validation process produces concrete outputs that can be directly compared between the Triton and PyTorch implementations.

Returning to the softmax example, Figure 1e shows the test code generated by the LLM for the Triton implementation. The generated test defines a helper function, `run_test_case(x)`, which executes the Triton kernel and returns both the input and output tensors. Three random input tensors(small, medium, and large) are then created with varying dimensions to test the kernel under different computational loads. Each case is executed on the GPU, and the resulting input–output pairs are appended to a list of test cases. These stored results form the reference outputs that are later used to verify the correctness of the translated PyTorch implementation.

In the second step, we pair the PyTorch translation with the previously generated Triton test cases to verify functional equivalence. The inputs of the step requires both the translated PyTorch code and the generated Triton test cases in the previous step. As shown in Figure 6, the prompt explicitly instructs the LLM to execute the PyTorch implementation using the exact same input tensors as those used in the Triton tests, rather than generating new ones. The LLM is also directed to reuse all existing variable names and maintain the original test structure, simulating a scenario in which the Triton tests have already been executed and their results remain in context. The generated test code then runs both the Triton and PyTorch implementations sequentially, comparing their outputs element-wise to confirm numerical equivalence up to a small numerical error.

Looking back to the softmax example, Figure 1c shows the PyTorch test code automatically produced for this step. The test executes the translated PyTorch softmax function on the same three input tensor that were generated in Figure 1d. For each case, the outputs of the PyTorch function are compared against those from the Triton kernel using `torch.allclose()` with a tolerance of 10^{-5}. This validation ensures that both implementations produce consistent numerical results across a range of input sizes, demonstrating the correctness of the translation.

5 Evaluation

We evaluated our framework on a suite of over 100 benchmarks spanning Triton tutorials[8], Mamba[3], Liger[5], and TorchInductor kernels[6]. Benchmarking was conducted on an NVIDIA Tesla T4 GPU for both Triton and PyTorch implementations, and on an Intel Gaudi2 accelerator for PyTorch execution. The Gaudi2 system(HL-225) is equipped with 24 Tensor Processing Cores (TPCs), 2 Matrix Multiplication Engines (MMEs), 48 MB of on-chip SRAM, and 96 GB of HBM2E memory[4]. The software environment for Gaudi2 used hl-gaudi2-1.18.0-fw-53.1.1.1, running on Ubuntu

24.04 LTS Docker with Habana PyTorch 2.4.0. For comparison, experiments on the NVIDIA Tesla T4 used PyTorch 2.8.0 (CUDA 12.6, cuDNN 9.1) on Linux 6.6.97+, with a 16 GB GDDR6 GPU containing 2 560 CUDA cores.

For code translation and verification, we employed LLMs including Gemini and GPT-o1 to generate and verify the PyTorch equivalents of Triton kernels.

5.1 Benchmark Suites

We evaluate TACT on a diverse suite of over 100 Triton kernels drawn from four major sources: Triton Tutorials, Mamba, Liger, and TorchInductor. Table 1 summarizes the qualitative characteristics of these benchmark suites, including the number of kernels, average lines of code (LOC) per kernel, the number of distinct Triton functions used, and the average number of Triton function calls per kernel, and average number of args per kernel.

The tutorial benchmarks are drawn from the Triton documentation, which and serve as an examples illustrating how to write and execute custom GPU kernels. In parallel, we also consider Mamba, a recently introduced sequence model that offers an alternative to Transformers by replacing attention mechanisms with state space models for more efficient long-sequence processing.

The Liger kernels are a collection of optimized Triton implementations designed to accelerate large-scale deep learning training. These kernels are handwritten in Triton to exploit fine grained GPU parallelism, reduce memory overhead, and improve throughput compared to baseline PyTorch implementations. We randomly selected 5 kernels from a suite of benchmarks being JSD, geglu, group norm, kldiv, and triton_qwen2vl_mrope. JSD is a method of measuring similarity between the probability distributions. It mainly involves elementwise operations, logarithms, and reductions. geglu is a variation of GELU activation where part of the input undergoes GELU and the other part acts as a gate. Group Norm is a normalization technique that divides channels into groups and computes normalization statistics per group. Kldiv is another divergence measure used as a loss function.

Finally, TorchInductor kernels differ from the previous suites in that they are not handwritten; instead, they are automatically generated by PyTorch's compiler backend during model execution. These fused kernels are produced dynamically to match the shapes and operations of the model graph, representing a realistic and diverse set of workloads emitted by an end-to-end deep learning compiler.

5.2 Translation Ability

Across all benchmark suites, TACT successfully translated all source Triton kernels into functionally equivalent PyTorch implementations. For the Liger kernels in particular, we further measured the number of translation iterations required across different LLMs (Table 2). Because each LLM query incurs computational cost and latency, we tracked the number of attempts needed to produce correct translations for five representative Liger kernels using Gemini 2.0 Pro, Gemini 2.5 Pro, and GPT-o1. In every case, the models achieved success within five attempts, with stronger models such as Gemini 2.5 Pro and GPT-o1 typically succeeding on the first try.

(a) Triton Tutorial

Benchmark	Triton	PyTorch
Softmax	52 µs	215 µs
Matmul	477 µs	12.6 ms
Dropout	46.7 µs	64.2 µs
LayerNorm	989 µs	1.13 ms

(b) Mamba kernels

Benchmark	PyTorch	Triton
Selective_state	3.46 ms	624 µs
ssd_state_passing	30.2 ms	731 µs
chunk state	1.37 ms / loop	1.02 ms

Figure 7: Runtime comparison of Triton vs. PyTorch implementations for Triton Tutorial and Mamba on NVIDIA T4 GPU setup mentioned above.

Each iteration in TACT comprises four distinct LLM queries, as described in the system overview. While we did not explicitly profile runtime, each query was capped at a two-minute timeout, giving a theoretical upper bound of eight minutes per iteration. In practice, however, the average turnaround time per iteration was closer to two minutes. Since each of the five attempts was independent and did not rely on intermediate outputs or feedback from previous iterations, they could, in principle, be executed in parallel for optimization. However, in our experiments, we performed iterations sequentially assuming the previous one failed.

The primary errors observed in the Liger kernels were related to output mismatches and tensor size inconsistencies, typically caused by subtle logical discrepancies in the generated code. One common issue arose when kernels returned multiple outputs. PyTorch occasionally produced these outputs in a different order than the original Triton implementation, leading to incorrect comparisons during validation. Interestingly, such inconsistencies often resolved themselves through additional sampling, as repeated iterations frequently yielded functionally correct variants. As for Triton tutorial and Mamba, no errors were encountered during translation and were correct within 1 iteration.

For TorchInductor kernels, the two dominant failure modes were compilation errors and runtime errors. Compilation errors primarily stemmed from updates in Triton's API such as changes to function signatures that introduced new arguments. Runtime errors, on the other hand, were most commonly due to CUDA out-of-memory exceptions, which could be mitigated through feedback-driven adjustments to reduce random input tensor sizes. This specifically only happened with the NVIDIA T4 GPU because of the limited memory avaliable.

5.3 Kernel Performance

We now analyze the performance across all benchmark suites. The Triton Tutorial and Mamba kernel experiments serve as our initial evaluation of the framework's ability to translate Triton kernels into functionally equivalent PyTorch implementations. As shown in Figure 7, Triton consistently delivers significantly faster GPU execution, particularly for compute intensive operations such as matrix multiplication, compared to the higher level PyTorch abstractions. For the Triton Tutorial suite, we observe speedups ranging from 1.14x to 26.4x, while the Mamba kernels achieve improvements of 1.34x to 41.31x. Although these benchmarks primarily serve as a proof of concept, they demonstrate the substantial performance potential of preserving Triton's low-level optimizations even after

Table 1: Qualitative characteristics of the benchmark suites used for evaluation. LOC = Lines of Code.

Suite	# Kernels	Avg. LOC	Unique Triton Funcs	Avg. Triton Func Calls	Avg. Args
Triton Tutorials	4	53.25	16	11.2	8.4
Mamba	3	138.7	12	22.8	40.8
Liger	5	50.5	9	5.8	13.2
TorchInductor	101	52	17	22.4	10.4

Table 2: LLM Translation Attempts per Kernel Type

Kernel	Gemini 2.0 Pro (Experimental)	Gemini 2.5 Pro	GPT-o1
JSD	3	1	1
geglu	1	1	1
group norm	5	2	1
kldiv	1	1	1
triton_qwen2vl_mrope	3	1	1
Total	**13**	**6**	**5**

translation, highlighting a clear path for extending this approach to additional hardware backends such as Intel Gaudi.

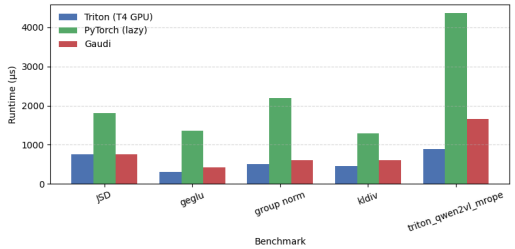

Figure 8: Runtime comparison of Triton (T4 GPU), PyTorch (lazy), and Gaudi implementations across Liger benchmarks.

Examining the results for Liger in Figure 8, we observe that Triton on the NVIDIA T4 GPU outperforms PyTorch by an average factor of approximately 3.7x. This performance gain is expected, as Triton's hand optimized kernels exploit low-level CUDA features such as hardware aware tiling and memory coalescing tailored to the kernel's structure, rather than incurring the overhead required to support dynamic shapes and generalized execution in PyTorch. When comparing PyTorch on T4 to PyTorch on Gaudi, the Gaudi backend shows an average 2.8x improvement, reflecting the efficiency of its execution environment. Comparing Triton on T4 against PyTorch on Gaudi yields a more modest 1.3x performance advantage for Triton. However, this should not be interpreted as a definitive comparison between the two hardware platforms. A conclusive assessment would require a broader set of benchmarks and a deeper exploration of hardware-specific optimizations, which are beyond the scope of this initial study.

Lastly, we examine the TorchInductor kernels, whose performance trend differs from that of the Liger benchmarks. In many cases, the Triton kernels performed worse than their PyTorch counterparts on both Gaudi and the T4 GPU. On average across all

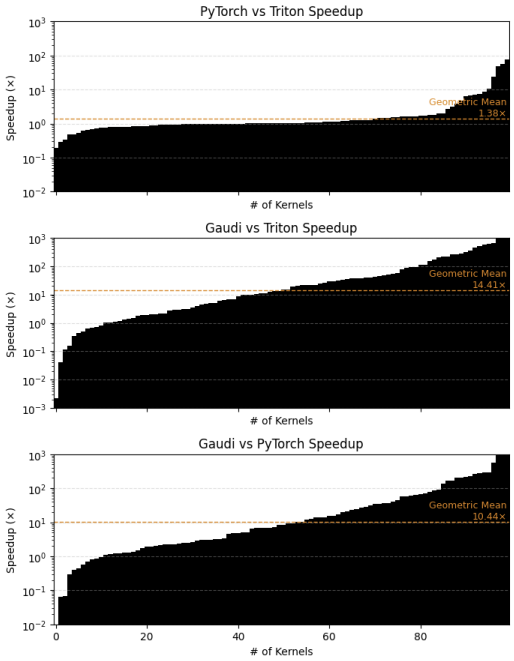

Figure 9: Runtime speedup performance between PyTorch on T4 vs Triton(top), Pytorch on Gaudi vs Triton(middle), Pytorch on Gaudi vs PyTorch on T4 for the TorchInductor benchmarks. There are 100 kernel benchmarks and are sorted from lowest speedup to highest speed up.

TorchInductor benchmarks, PyTorch on T4 is 1.38x faster than Triton on T4 and PyTorch on Gaudi is 14.41x faster than Triton on T4. This difference is expected, as these Triton kernels were not handcrafted Triton implementations but rather automatically generated by TorchInductor as intermediate code. Consequently, the

generated Triton kernels inherit TorchInductor's optimization assumptions and scheduling parameters tuned for a different device, rather than the T4 GPU used in our experiments. As a result, kernel tuning defaults (such as fixed num_warps, num_stages, and tiling strategy) may not align with with the underlying GPU architecture or its memory access patterns. This mismatch led to substantial slowdowns, as the LLM generated test cases were not running at the hardware-optimized configurations. More importantly, when comparing PyTorch performance across hardware, the Gaudi accelerator consistently outperforms the T4 GPU (Figure 9), highlighting the strength of moving to different accelerators.

6 Related Works

Recent advances in LLMs have sparked growing interest in their ability to generate performance optimized code, particularly for high performance computing and GPU kernel development. Ouyang et al. introduced KernelBench[9], a comprehensive benchmark that systematically evaluates LLMs on their capability to write efficient and correct GPU kernels from PyTorch workloads. KernelBench formalizes the challenge of kernel generation by assessing both correctness and performance, revealing that even state-of-the-art reasoning models struggle to match or surpass hand optimized PyTorch implementations on most tasks. While KernelBench primarily explores translating from high level PyTorch code to low level CUDA kernels, our work translates in the opposite direction, from Triton kernels to functional equivalent PyTorch implementations.

Another closely related effort is TritonBench[7], a benchmark specifically designed to evaluate LLMs on generating Triton GPU operators. Unlike conventional code benchmarks that primarily assess functional correctness, TritonBench emphasizes performance-aware evaluation by measuring both correctness and GPU execution efficiency on NVIDIA hardware. While TritonBench focuses on the forward direction by using LLMs to generate Triton kernels from PyTorch, our work (TACT) again explores the reverse direction: translating existing Triton implementations back into PyTorch to evaluate correctness and functional equivalence.

In parallel, Bhatia et al. proposed LLMLift[1], a framework for verified code transpilation using LLMs. LLMLift extends the concept of verified lifting, where code in a general-purpose language is automatically translated into a domain-specific language while formally proving semantic equivalence. By leveraging LLMs for proof synthesis, LLMLift demonstrates that neural models can outperform symbolic synthesis based tools in coverage and efficiency while providing formal correctness guarantees.

7 Conclusion

In this work, we demonstrated TACT, a framework for translating Triton kernels into PyTorch implementations using LLMs. Our approach combined test-case generation with a two-stage translation process, ensuring functional equivalence and enabling execution across hardware backends that do not natively support Triton. Through evaluations on more than one hundred benchmarks, we showed that the framework can reliably produce correct translations in only a few attempts and, in many cases, preserve or even improve performance.

Looking ahead, there are several promising directions for future work. Extending evaluations to more recent hardware, such as NVIDIA A100 or H100 GPUs, will provide a clearer picture of how the translation performs on state-of-the-art accelerators. Another direction is to incorporate better feedback mechanisms into the translation process, such as leveraging runtime errors or hardware-specific hints to guide LLM-based code generation toward more efficient kernels. Together, these extensions would further strengthen the case for TACT as a practical pathway to portable, high-performance deep learning kernels.

References

[1] Sahil Bhatia, Jie Qiu, Niranjan Hasabnis, Sanjit A. Seshia, and Alvin Cheung. 2024. Verified Code Transpilation with LLMs. arXiv:2406.03003 [cs.PL] https://arxiv.org/abs/2406.03003

[2] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. 2024. Large Language Monkeys: Scaling Inference Compute with Repeated Sampling. arXiv:2407.21787 [cs.LG] https://arxiv.org/abs/2407.21787

[3] Tri Dao and Albert Gu. 2024. Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality. arXiv:2405.21060 [cs.LG] https://arxiv.org/abs/2405.21060

[4] Habana Labs. 2025. *Intel Gaudi AI Accelerator Documentation.*

[5] Pin-Lun Hsu, Yun Dai, Vignesh Kothapalli, Qingquan Song, Shao Tang, Siyu Zhu, Steven Shimizu, Shivam Sahni, Haowen Ning, Yanning Chen, and Zhipeng Wang. 2025. Liger-Kernel: Efficient Triton Kernels for LLM Training. In *Championing Open-source DEvelopment in ML Workshop @ ICML25.* https://openreview.net/forum?id=36SjAIT42G

[6] Intel Corporation. 2025. Intel XPU Backend for Triton. https://github.com/intel/intel-xpu-backend-for-triton.

[7] Jianling Li, Shangzhan Li, Zhenye Gao, Qi Shi, Yuxuan Li, Zefan Wang, Jiacheng Huang, Haojie Wang, Jianrong Wang, Xu Han, Zhiyuan Liu, and Maosong Sun. 2025. TritonBench: Benchmarking Large Language Model Capabilities for Generating Triton Operators. arXiv:2502.14752 [cs.CL] https://arxiv.org/abs/2502.14752

[8] OpenAI. 2025. Triton Tutorials: Getting Started with GPU Programming. https://triton-lang.org/main/getting-started/tutorials/.

[9] Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. 2025. KernelBench: Can LLMs Write Efficient GPU Kernels? arXiv:2502.10517 [cs.LG] https://arxiv.org/abs/2502.10517